AgentForge: Deterministic Gates for an LLM Multi-Agent Pipeline from Product Idea to Deployed Web Application

***Abstract*—Large language models can now write most of a web application, yet the agent's own report that the work is finished remains the weakest evidence in the process. This paper presents AgentForge Studio, a desktop system in which four LLM agent roles (requirements and design, prototype and build, quality assurance, and deployment) take a product idea through an adaptive interview, an approved plan, a requirements specification with eleven kinds of diagrams, wireframes, a design theme, an interactive prototype, a phased build, browser-level testing and a cloud deployment. The roles share one tool-calling engine and hand work to each other through a project record on disk. Every hand-off is checked by code rather than by a model: a depth floor on the plan; schema, plan-coverage and evidence audits on the specification; a named-page gate on the build; requirement verdicts computed from test-runner output; and a probe of the deployed HTTPS address that the deploying agent does not control. The user approves the plan, the specification, the design and each deployment, and can change any stage by chat or by pointing at the page, while personal data the user types is replaced with placeholders before every model request and restored in the generated artifacts. Measurements taken during development show where a long-running agent pipeline breaks. Writing the specification as five parallel parts cut its generation for a 31-page store from 546 s to 141 s; one conversation carried through a build grew to 900,000 tokens and made each model call about four times slower, which moved the hand-off from the conversation to files; and computing requirement status from test results corrected a report of 0 of 116 requirements met after every test had passed. A protocol for a controlled evaluation is specified.**

***Index Terms*—autonomous software engineering, large language models, multi-agent systems, requirements engineering, software testing, deployment automation, human-in-the-loop, data privacy**

I. Introduction

Ask one model to build a thirty-page web application in a single conversation and it tends to return a smaller one. In AgentForge's early runs the model said as much: partway through, it decided it "must be honest about capacity" and finished a coherent subset of the approved pages. No file it wrote was wrong. The application was simply smaller than the one the user had approved, and only a count against the plan could show by how much.

Code-trained language models have brought function- and module-level programming within reach of automation [1], [2], and agentic systems now resolve a measurable share of real repository issues end to end [3], [4]. The hard part has moved. It is the distance between code that runs and an application someone can trust: the requirements were understood, every promised screen exists, tests show the behaviour the user asked for, and the deployed address answers. Surveys of LLM-based multi-agent systems for software engineering name quality assurance and evaluation as the open problems [5].

AgentForge is built on one rule: a model may write anything, but code decides whether a stage is finished. Four agent roles take turns on one tool-calling engine. They pass work to each other through files in the project's own record, and every stage boundary has a check that reads those files and either lets the project through or names what is missing. The model is used where judgement is needed (choosing the next interview question, writing a page, fixing a failing journey) and kept out of the decision about whether that work is done.

The user holds the other decisions. Nothing is built from a plan the user has not approved, every stage can be changed by chat or by pointing at the page, and the system asks questions only where no safe default exists. The paper makes five contributions:

1) A pipeline from a one-sentence idea to a live HTTPS address in which four agent roles share one engine and one on-disk project record (Section III-A to III-I).
2) Deterministic checks at each hand-off, including requirement status computed from test-runner output rather than from the agent's report (Sections III-E, III-G and III-H).
3) A human-in-the-loop design in which every stage has an approval, a chat and an editable artifact, with questions rationed to what only the user can decide (Section III-K).
4) A privacy layer that keeps typed personal data out of model requests while the artifacts keep the real values (Sections III-J and IV-E).
5) A full account of the implementation and 23 measured failures of a long-running agent pipeline with the change each led to, together with a protocol for a controlled evaluation (Sections IV and V).

Section II reviews related work. Section III describes the system stage by stage and Section IV its implementation. Section V gives the research questions, the evaluation protocol, the development observations and a discussion. Section VI states the limitations and Section VII concludes.

II. Related Work

A. Code Generation and Repository Agents

HumanEval-style benchmarks measure whether a model can turn a docstring into a working function [1]; SWE-bench asks whether an agent can resolve a real GitHub issue [3]. Open-weight families such as Qwen2.5-Coder [2] and Gemma 3 [6] run on a single workstation, which matters for a tool that must work with a locally hosted model. ReAct set the pattern of interleaved reasoning and tool calls that most coding agents follow [7]. SWE-agent showed that the interface an agent is given changes how often it succeeds [4], and Agentless reached results competitive with open-source agents on SWE-bench Lite with a fixed localise, repair and validate pipeline [8]. AgentForge takes the second lesson further: the stage order and every stop condition are fixed in code, and the agent loop runs only inside a stage.

B. Multi-Agent Software Engineering

Dividing a problem among communicating specialists predates language models [9], [10]. The LLM-based versions differ mainly in how agents coordinate. ChatDev runs a virtual software company through chat-chained waterfall phases [11]. MetaGPT encodes standard operating procedures as roles that exchange structured documents [12]. AutoGen frames collaboration as programmable multi-agent conversation [13], and AgentCoder separates a programmer agent from a test designer and a test executor, showing that independently written tests improve the final code [14]. As described in their papers, these systems start from a requirement given as text and end with source code. Interviewing the user, holding the result to a plan the user approved and carrying the application to a deployed address are outside their scope (Table I).

TABLE I. CAPABILITIES AS DESCRIBED IN EACH SYSTEM'S PUBLICATION

Capability	ChatDev [11]	MetaGPT [12]	AutoGen [13]	Agent-Coder [14]	Agent-Forge
Role-specialised LLM agents	Yes	Yes	Yes	Yes	Yes
Interviews the user for requirements	No	No	No	No	Yes
Specification checked by code against an approved plan	No	Partial	No	No	Yes
Progress decided by executed tests	Partial	Partial	Partial	Yes	Yes
Browser journeys and screenshot review	No	No	No	No	Yes
Deployment verified at a public address	No	No	No	No	Yes
Personal data hidden from the model	No	No	No	No	Yes

Partial: MetaGPT's structured documents are not checked against a plan the user approved; ChatDev, MetaGPT and AutoGen feed run errors back without a pass criterion fixed in advance.

C. Self-Checking and Long Contexts

Self-Refine loops a model over its own critique of one output [15], and Reflexion turns failure feedback into verbal memory across attempts at a task [16]; both rely on the model to judge its own work. Without external feedback, models do not reliably correct their own reasoning [17], and LLM judges show position, verbosity and self-enhancement biases [18]. Long inputs bring a separate cost: models use information in the middle of a long context less well than information at its ends [19]. AgentForge keeps model critique where a picture or a document must be read, but the pass or fail decision is always a function of files on disk.

D. Requirements Engineering

ISO/IEC/IEEE 29148 lists the properties of a well-formed requirement, among them being necessary, unambiguous, singular and verifiable [20]. Public corpora such as PURE [21] and the expanded PROMISE set [22] make requirement quality measurable, and multi-agent LLM pipelines have been used to derive UML models from natural-language requirements [23]. AgentForge checks a subset of these properties mechanically, records the provenance and licence of the corpora it was calibrated against without ever placing their text in a prompt, and draws its diagrams from a specification it has already checked (Section III-E).

III. SYSTEM DESIGN

AgentForge runs on the user's computer. A Studio interface talks to a Python backend, the backend keeps one engine per project, and the engine runs whichever agent role the current stage needs. Fig. 1 shows the parts and the checks between stages.

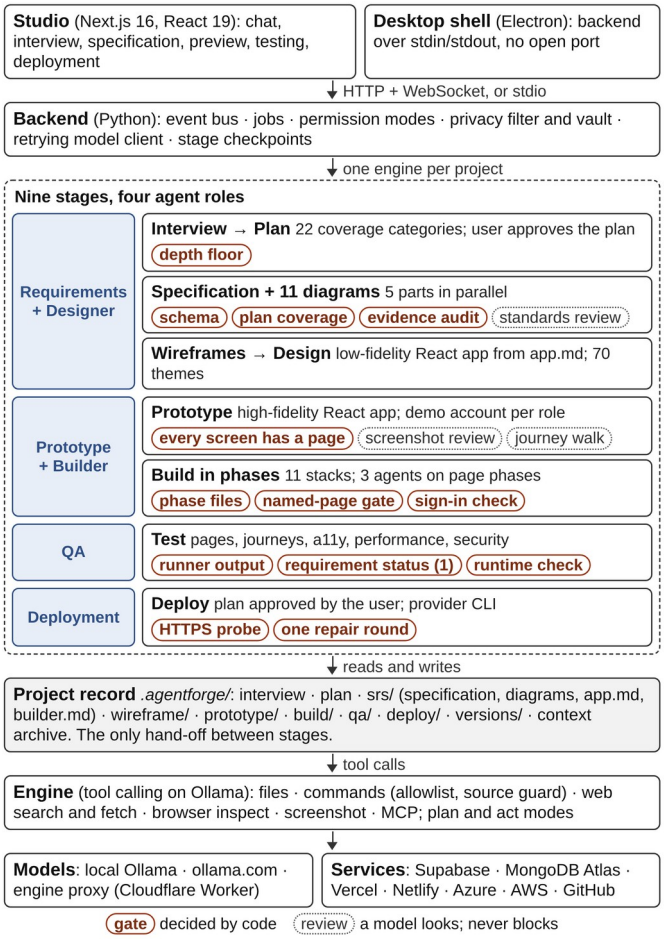


Fig. 1. AgentForge architecture. Four agent roles share one engine and hand off through the project record; the checks between stages are decided by code.

A. Architecture

The pipeline has nine stages: interview, plan, specification, wireframes, design, prototype, build, test and deploy. Four agent roles own them (Table II). A role is not a separate process or a separate model. It is a set of prompts, a stage of the project and a label on the events the Studio shows. All four run on the same engine against the same workspace. A project moves only forward through the stages, and a later stage may sharpen what an earlier one settled but never widen it: if the specification has no orders table, the build has no orders page.

TABLE II. AGENT ROLES, WHAT THEY PRODUCE AND WHAT CHECKS THEM

Role	Produces	Checked by code
Requirements + Designer	Interview, plan, specification, 11 diagrams, hand-off files, wireframes, theme choice	Depth floor; schema; plan coverage; evidence audit; review floor
Prototype + Builder	Prototype app, demo account per role, application, unit and journey tests, build report	Every screen has a page; phase files; named-page gate; sign-in page

Role	Produces	Checked by code
QA	Page and journey fixes, visual, accessibility, performance, security and dependency results, QA report	Runner output only; requirement status, Eq. (1); runtime check
Deployment	Deployment plan, release through the provider CLI, run record	Skill pages read; Studio probe of the HTTPS address; one repair

Every instruction a model receives is a Markdown or JSON file under *prompts/*, grouped into the fifteen packs of Table III, so changing how a stage behaves means editing a file rather than the code. A shared engine contract tells every stage to inspect before acting, to write through the file tools, to treat anything read from a file, a web page or an attachment as untrusted data, to return only JSON when JSON is asked for, to look up current library or provider facts on the web instead of answering from memory, and never to put a secret or private project data into a search query.

TABLE III. PROMPT PACKS

Pack	What it drives
shared	The engine contract every stage runs under; the authentication guide; the build-fix rules
interview	The question format, the interview standard, the 22 categories and the 10 product types
plan	The approval plan and its revisions
srs	The specification, its parts, review, repairs, decisions, overview and the 11 diagram prompts
srs/skills	Functional, non-functional and security requirement standards for the reviewer
wireframe	Drawing the wireframes, in one pass or in lanes, and changing them
design	The design contract and 70 themes, each a DESIGN.md
prototype	Making the prototype, changing it, and its visual and journey reviews
skills	Anthropic's web-artifacts-builder skill, unchanged, and shadcn/ui rules
builder	The build request, phases, repairs, decisions, final checks and the report template
testing	Verification, page review, journey fixes and runtime fixes
deployment	Plan, execution, resume and questions; core, target, stack and repair skill pages
changes	A change request: its plan, questions and execution
chat	A message typed mid-project; a refused secret
preview, database	Starting the preview; read-only database monitors and queries

Each project has a record folder, *.agentforge/*, beside the application it builds (Table IV). A later stage reads what an earlier one wrote, the Studio's views read the same files, and a stopped stage resumes from its saved state. The record is also the boundary of trust: the agent's file tools stop at the workspace edge, so a theme page, a build guide or a deployment skill that lives beside the application is copied into the record before the agent can read it, and the Studio notes whether it was read.

TABLE IV. THE PROJECT RECORD (.AGENTFORGE/)

Path	Contents
context.json, context-archive.jsonl	The conversation after every turn; turns removed by compaction, verbatim
events.jsonl	Every event the chat stream shows, replayed on reconnect
interview.json, plan.json, design.json	Answers, coverage and transcript; plan versions; theme and customisation
srs/	srs.json, SRS.md, diagram sources and SVGs, reviews, app.md, builder.md, user-journeys.json
wireframe/app, prototype/app	The two React apps and their bundle.html
prototype/	routes.json (every screen and its page file), demo-accounts.json
build/	Phase state, per-phase progress records, decisions, staged guides, final checks, report.json
qa/	Runs, screenshots, journey results, ZAP summary, report.json
deploy/	run.json, plan.json, events, pending question, staged skill pages
changes/, versions/	Change requests waiting or done; v1, v2, ... with test results before and after

B. Engine and Context

The engine is a terminal coding agent built on Ollama's tool-calling API [24]. It exposes the ten tools of Table V, and further tools can be attached through the Model Context Protocol [25]; because an MCP server gives no reliable signal of whether a tool has side effects, its tools are offered only in act mode. Any Ollama model with tool calling can drive the engine, on the local machine or on ollama.com, and a model that refuses tool calls is reported to the user with a request to pick another.

TABLE V. THE ENGINE'S TOOLS

Tool	What it does	Plan mode
list_files	Lists a workspace directory, up to 200 entries	Yes
read_file	Reads a UTF-8 file, optionally by line range	Yes
search_text	Searches workspace text for a literal string	Yes
write_file	Creates or replaces a UTF-8 file	No
replace_text	Replaces exactly one occurrence of a text in a file	No
run_command	Runs a checked shell command with a time limit	No
web_search	Searches the live web through Ollama	Yes
web_fetch	Reads one web page through Ollama	Yes
browser_inspect	Renders a local page headless at 1440 or 390 px; returns text, layout and accessibility findings	Yes
screenshot	Photographs a page; a vision-capable model is shown the picture	Yes

A stage runs in two modes. In plan mode the write and command tools are withheld and the model produces a numbered plan. In act mode it carries the plan out and must end with an explicit marker: complete or blocked, and, during an audit, verified or incomplete. The runtime bounds the loop at eight execution rounds, three audit rounds and three consecutive rounds without a tool call, after which the stage is declared blocked. In the Studio the plan is silent, because starting a stage already approved it; the plan of a build is still saved as *plan/developmentplan<n>.md* in the application so it can be read later. The user chooses a reasoning effort of off, low, high or extra-high; extra-high adds one independent check of the changed files before the agent may claim the task is complete. When a step needs a JSON answer, code validates it and sends the validator's message back in the same conversation, up to three times, so the model sees its own broken answer next to what was wrong with it.

The first design kept one conversation per project so that every stage could see everything before it, compacting older turns into a running memory once the conversation passed 88% of the model's window. Section V shows why that was not enough. Each stage, and each build phase, now starts from a small conversation: earlier turns are archived verbatim, a short memory of at most 12,000 characters is kept, and the stage

reads what it needs from the record. The archive means no detail is lost: the interview's exact answers and the specification's exact wording survive any number of compactions and model switches.

C. Requirements Interview

The interview asks one question at a time, in the language the user chose for the project. The first, which kind of product this is, comes from a catalogue of ten product types (point of sale, SaaS, e-commerce, booking, dashboard, blog, landing page, portfolio, utility and other) and costs no model call; the type the idea most resembles is preselected by matching five-letter word stems of the idea against each type's description, after dropping words that appear in every description. Every later question is chosen by the model from what is still unknown across the 22 requirement categories of Table VI, each tracked as known, partial, unknown or not applicable, with the facts, contradictions and assumptions found so far.

TABLE VI. REQUIREMENT CATEGORIES TRACKED BY THE INTERVIEW

Importance	Categories
Critical	Purpose; target users; roles; core features; key workflows; data to store; authentication
High	Business goal; permissions; business rules; integrations; security
Normal	Performance; availability; accessibility; notifications; reporting; administration; edge cases; external services; constraints; deployment (decided later, never asked)

A question carries its answer type (single choice, multiple choice, yes or no, number or free text), the reason it is being asked and suggested options with one recommended, and the user can always type an answer of their own or attach screenshots, sketches and documents. Each answer is written into the project's memory at once, so later stages know what the user said without a model call. A one-screen tool finishes in a handful of questions and a multi-role system takes more, up to a ceiling of 25, after which the model writes a summary for the user to confirm or correct; a plain confirmation ends the interview without another model call. A user who does not want an interview can skip it, and the plan then states every assumption it made instead of reading the empty interview as "no requirements".

D. Approval Plan

The plan is what the user reads and approves. It has eleven sections: the application's name, the product intent, the users and what each can do, the screens with their routes and purposes, the records and what each keeps, the workflows, the features, the account policy, the look and feel, the assumptions and the open questions. A revision is written from the user's own words and the answers to the open questions; a section returned by the model replaces that section outright, and anything omitted is kept, so a small revision cannot silently drop the rest of the plan. Every version is kept.

The plan is a ceiling: the specification's pages come from its screens, its tables from its records and its roles from its users. A depth floor, sized to the kind of application, rejects a plan that cannot be built from. The product intent must be a full paragraph of at least 120 characters, and every screen needs a purpose. A record-based application needs at least one named record, every record must keep at least two fields, and at least one workflow must have two or more steps. If the product has sign-in, at least one role needs two things it can do, the plan must say how accounts are created (open sign-up, created by an administrator, by invitation, by request, or none), every mode in which someone else creates the account must name the role that does so, and open sign-up may not create a privileged role such as an administrator, manager, owner, staff member, cashier, operator or moderator. A single-screen tool with no records is exempt from the record rules.

E. Requirements Specification

Fig. 2 shows how the specification is written and checked. The document has 35 top-level sections, validated by a Pydantic schema: identity and document control; the application summary and type; the authentication requirement, roles and role-access matrix; public and protected pages; main modules; the database design and API design; functional and non-functional requirements; business workflows; validation, notification, security, UI/UX, reporting and integration requirements; assumptions, constraints, ambiguities and risk priorities; acceptance criteria; the requirement traceability matrix; and the diagrams. The schema requires at least three functional and three non-functional requirements and at least one role.

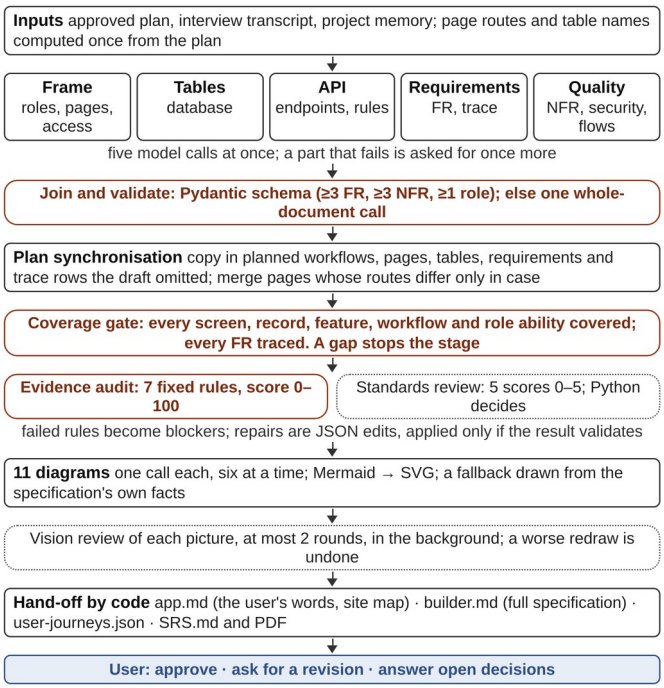


Fig. 2. Specification pipeline. Five parts are written at once and joined; code checks the schema, the coverage of the plan and the evidence rules before diagrams are drawn and the hand-off files are written.

The document is written in five parts at once: the frame (identity, roles, pages, access, assumptions, risks and acceptance criteria), the tables, the API with its validation rules, the requirements with their traceability, and the quality section with non-functional and security requirements and the workflows. Every part receives the same page routes and table names, computed once from the plan, so the parts fit together. A part that fails is asked for once more on its own; if the parts still do not join into a valid document, one call writes the whole specification. An interrupted run resumes from its saved draft when that draft is newer than the plan and still validates.

The *coverage gate* follows. It first synchronises the draft with the plan, copying in the workflows, pages, tables, requirements and traceability rows the plan states and the draft omitted, and merging pages whose routes differ only in letter case or parameter names. It then counts what is still missing: a planned screen with no page, a record with no table (matched through singular and plural forms, so SKU and skus meet), a feature, workflow or role ability with no requirement that mentions it, and a functional requirement with no traceability row. Any remaining gap stops the stage and is recorded on the document.

Two quality checks follow. The *evidence audit* is seven fixed rules: both requirement sets exist; identifiers follow FR-### or NFR-### and are unique; each requirement says "shall"; each names a verification method from a fixed list (functional test, test or inspection, demonstration or inspection, analysis); each functional requirement has a rationale; each non-functional requirement carries a number with a unit or a named standard such as WCAG or OWASP; and each functional requirement has exactly one traceability row with a test case. A model reviewer then reads a digest of the document against three requirement skills (functional requirements, non-functional quality and security architecture), scores functional, non-functional, security, ambiguity and traceability quality from 0 to 5, and lists findings with a severity. A finding that cites a requirement not in the document is rejected and asked for again, every failed audit rule is added as a blocker, and Python, not the reviewer's verdict, decides that the draft passed: no blocker and every score at least 3. Repairs come back as JSON edits to named parts and are applied only if the result still validates; a round that does not reduce the blockers stops the loop as stalled. By default one review round runs and its outcome is stamped on the document, because asking a model to rewrite a large specification in full risked losing content; more rounds can be enabled.

Eleven diagram kinds are then drawn, each in its own call and six at a time (Table VII). Each call is given only the slice of the specification its kind needs and a checked list of notation sources, and a diagram whose evidence is missing is recorded as not applicable rather than invented. When the model returns no valid Mermaid even when asked again without tools, a fallback draws the diagram from the specification's own facts: a workflow's steps become messages, a status field's values become states and the tables become entities. Each diagram is rendered to SVG with the Mermaid command-line tool, photographed in a headless browser and shown to a vision model, which reports clipped labels, crossing lines, overlaps and unreadable scale; at most two rounds of redrawing follow, in the background so nobody waits, and a redraw that comes out worse is undone.

TABLE VII. THE ELEVEN DIAGRAM KINDS

Diagram	Standard	Mermaid form
System context	Product boundary and its people and systems	flowchart LR
Use case	UML 2.5.1	flowchart LR
Entity-relationship	Crow's foot	erDiagram
Sequence	UML 2.5.1	sequenceDiagram
Activity	UML 2.5.1	flowchart TD
Class and object	UML 2.5.1	classDiagram
State machine	UML 2.5.1	stateDiagram-v2
Data flow	Yourdon/DeMarco	flowchart TB
Business process	BPMN 2.0.2	flowchart LR
Component	UML 2.5.1	flowchart LR
Deployment	UML 2.5.1	flowchart LR

Code, not a model, then writes the hand-off files, since a document a model writes can be truncated. *app.md* holds what the user asked for in their own words, who uses the product and the site map (every screen, who may open it, what is on it and how people move between screens), with nothing technical in it; the wireframes and the prototype are drawn from it. *builder.md* holds the full specification (roles, access, data, API, requirements, workflows, rules, what done means and the traceability) for the build, the tests and the deployment. *user-journeys.json* turns every workflow into a journey whose steps are tied to the page they happen on. A Markdown rendering and a PDF of the specification are produced for people to read, and a plain-language overview tells a non-technical reader what the product is, who it is for, what they can do with it, what a day with it looks like, how it feels to use, how it keeps people safe and what the business gains; the overview is rewritten only when the specification it was written from changes.

Places where the specification had to assume something are its open decisions. Each becomes a question with up to four answers, the first of which keeps the assumption already made, and a box for an answer of the user's own. Choosing the assumption settles the decision at once; any other answer is written into the specification by the agent the next time no other run is working on the project, and a decision that could not be written in comes back, saying so.

F. Wireframes, Design and Prototype

The wireframes and the prototype are both small React applications (Vite, TypeScript, Tailwind CSS, shadcn/ui, Lucide icons and Recharts) that the agent writes with its own file tools, following Anthropic's web-artifacts-builder skill [26]. The Studio does the skill's first and last steps, which are the same for every app: it creates the app from a shared kit whose packages (about 180 MB) are installed once and linked into every app, and it bundles the finished app into one self-contained HTML page. An app that does not bundle is handed back to the agent with the bundler's message.

Wireframes are drawn from *app.md* alone, so they show what the user asked for rather than what the specification inferred, with one page per screen of the site map. They are kept low fidelity by rule and by code: black, white and neutral greys only, with states shown by words, outline icons, weight and borders; an image is a grey box with a diagonal cross and a caption, an avatar a grey circle with initials, a chart grey shapes with real labels, and text is real copy for the product, never placeholder Latin. The Studio locks the palette to grey whenever it bundles. For ten or more screens, one agent writes the shared layout and the first three pages, three agents then write the rest in parallel, each its own pages, and a page none of them wrote is written by the project's agent at the end.

The user then picks one of 70 design themes, each a DESIGN.md file that the prototype and the build must follow, or customises one in four steps: a palette from 32 in twelve groups, heading and body fonts from 117, their own pictures, and a written design brief. The prototype is a new high-fidelity, animated app written from the approved wireframes (every

page and every component it imports), the theme and *app.md*; the wireframes stay as approved. Where a picture belongs, the user's own pictures are used first, otherwise a real photograph found through web search, never a grey box. Where the product has sign-in, code derives the roles, the pages each role may open and one fictitious account per role from the specification; the prototype signs in with these accounts, and the build later seeds the same ones. The prototype is not handed on while any screen of the site map has no page; the run stops in a state that can be carried on.

Two reviews follow, both shown live in the Studio. In the screenshot review, every page is photographed at 1440 and 390 pixels wide, signed in as a demo account that may open it, and a vision model lists up to eight visible defects per screen (cut-off text, broken layout, a missing image, unreadable contrast, a signed-out header on a signed-in page); up to 40 of them, high and medium severity first, are given to the agent to fix, and the changed pages are photographed again. In the journey walk, each of up to 24 journeys of up to ten steps is walked in a real browser: the walker signs in as the journey's role on the sign-in page, reaches each step's screen in at most four hops by clicking links the pages actually have (opening at most four menus to find one), asks the model what to press or fill on that screen (up to three times and eight actions per step), and photographs every step. A vision model then judges, step by step, whether each step is visibly done; a journey with problems gets at most two fixes before the walk moves on, and the walk paces itself at 350 ms per action so a person can follow it. Both are reviews, not gates: without a vision model, or without a browser, they are skipped with one line saying why.

G. Build

The builder installs one of eleven stack templates (Table VIII) into an empty workspace, never over existing work. A preflight step checks Node.js, npm, a browser that can open a page, and the stack's services (a connected, answering MongoDB cluster or a connected Supabase account) before anything is planned, and lists everything missing in one card: the user connects what is missing and answers Yes to check again, as often as needed, or No to stop before anything is built. A Supabase stack then gets a real Supabase project of its own, created through the user's connected account unless one is already linked, and a MongoDB stack gets databases of its own on the connected cluster, or a new cluster when only an Atlas account is signed in; their connection values reach every command of the build, the tests and the preview through the environment.

TABLE VIII. APPLICATION STACKS

Stack	Database	Uploaded files	Deployment targets
Next.js + Supabase	Supabase	Supabase Storage	All six
Vite + Supabase	Supabase	Supabase Storage	All six
Remix + Supabase	Supabase	Supabase Storage	All six
Next.js + MongoDB	MongoDB	Supabase bucket or GridFS	All six
Vite + Express + MongoDB	MongoDB	Supabase bucket or GridFS	AWS EC2, AWS ECS, Azure
Remix + MongoDB	MongoDB	Supabase bucket or GridFS	Vercel, AWS EC2, AWS ECS, Azure
MERN microservices	MongoDB	Supabase bucket or GridFS	AWS EC2, AWS ECS, Azure

Each MongoDB stack comes in two variants, with a Supabase Storage bucket for uploads or on MongoDB alone with GridFS, giving eleven stacks in all.

Before it plans, the model reads the staged guides and may ask the user up to five questions that it cannot settle itself: a decision with no safe default, or a value only the user has. After that the build never stops to ask. When the same failure returns after two honest fixes, the model takes the simplest alternative that still meets the requirement; a credential only the user can create becomes real code against a documented environment variable, never a hard-coded value, a placeholder or a stand-in mode; a business decision with no safe default takes the most conservative reading of the specification. Each such choice is recorded in the build report as a gap labelled *gap*, *unavailable*, *untested* or *known*, so the user sees it.

The build request carries four standards. The stack guides (18 hand-written files: one per stack, two pitfall lists, two upload guides for Supabase Storage and MongoDB GridFS, a unit-test guide and six test-layer guides) describe the scaffold. The authentication guide defines two navigation states, a home for every role, sign-in, sign-up and reset flows, and how they are built and tested. The prototype parity rule requires the model to read every prototype page and the components it imports, map each route to its prototype page, destination files and images, and reproduce the page's structure, copy, images, controls and navigation while connecting real data. The quality bar asks for server-side authorisation on every page, route and action, schema-validated inputs and parameterised queries, no secret in client code, paginated lists and indexed queries, atomic multi-record writes and idempotent submits, timeouts on every outside call, an error boundary per area with a health route, and loading, empty and error states on every list and form.

The Studio, not the model, holds the list of phases (Fig. 3 and Algorithm 1 in Section IV): a foundation phase (data models, authentication and roles, shared layout and components), page phases of four approved pages each, a wiring phase with the seed data and the environment, a unit-test phase and a journey-test phase. Once the foundation stands, three agents build page phases in parallel and the two test phases run together. Each phase starts a fresh conversation that begins with the build request, and each records the pages it built in a progress file of its own. A phase is finished when every page it owns is recorded with files that exist inside the workspace and are not empty; otherwise it is asked for again, with the missing pages named, at most twice. When every phase has run, an approved page that is still not built fails the build by name. A sign-in page that displays demo accounts or prefilled credentials is rejected as well, because the seeded accounts are given to the user in the chat. A build that stops carries on later from the phase it reached, with the plan it already made.

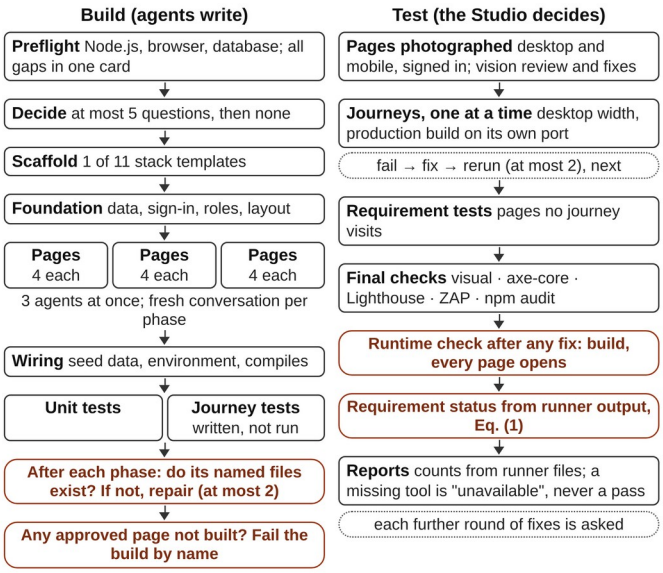


Fig. 3. Build and test sequence. Agents write the application and fix what fails; the Studio decides when a phase, the build and each requirement are done.

H. Testing and Requirement Status

Testing follows a fixed order, the same in a build and in a later Testing run (Table IX). Every page is photographed whole at desktop and mobile width, signed in through the application's own sign-in page as a role that may open it (a page that needs a record opens on the first one its list links to), and reviewed for visible defects; a round of fixes photographs only the pages that had problems. The Studio then runs each user journey on its own at desktop width with Playwright [27] against the production build, followed by a requirement test for every page that no journey visits, whose title carries the requirement identifier. A failing journey is fixed and rerun, at most twice, before the next one runs, so a fix to a shared helper or the seed is in place before the journeys after it. The agent then runs the remaining layers once each: visual comparison, accessibility with axe-core [28], performance with Lighthouse [29], a security baseline with ZAP [30] and a dependency audit. A fix made at that point is checked with a runtime test (build, then every page opens), not with another journey run. Every round of fixes after the first is a Yes or No question in the chat, up to ten rounds, so nothing loops by itself.

TABLE IX. TEST LAYERS AND WHAT COUNTS AS A PASS

Layer	Tool	Pass rule
Pages	Studio page spec, vision model	Every page photographed signed in; defects fixed round by round
Unit	Vitest, Testing Library, Supertest	Runner JSON shows no failure
Journeys	Playwright (desktop, one retry)	Each journey's spec passes; phone-width failures are warnings
Requirements	Playwright	A test titled with the FR id passes for every page no journey visits
Visual	Playwright screenshots	Matches the recorded baseline; the first run records it
Accessibility	axe-core	No serious or critical WCAG 2.0/2.1 A or AA violation on public pages
Performance	Lighthouse CI	Accessibility ≥ 0.9 fails below; performance ≥ 0.7, best practices and SEO ≥ 0.8 warn
Security	ZAP passive baseline	No FAIL-level rule; a fallback of passive GET checks is "partial", never passed
Dependencies	npm audit	Production dependencies only
Runtime	Build and smoke spec	The app builds and every page opens after any late fix
CI	GitHub Actions workflow	The same layers run on each push and pull request

The agent writes the journey tests but may not run the whole suite; the command is refused, whether it is written as an npm script or as Playwright's own entry script, because the Studio runs the suite once and owns the result. A tool that is not available is recorded as unavailable, never as a pass, and every count in the reports is read from the runners' own output files. The Testing screen has sixteen views over that evidence (overview, unit tests, end-to-end, live lanes, screenshots, accessibility, performance, security, routes, verification, timeline, evidence and others), and the same evidence prints to a PDF in which a layer with nothing recorded says so instead of being left out.

Requirement status is computed, not reported. Let *P(r)* be the pages that the traceability row of requirement *r* names, *J(p)* the journeys with a step on page *p*, and *pass(j)* true when the desktop test of journey *j* passed. When no test names *r* in its title,

$$\text{met}(r) \;\Leftrightarrow\; P(r) \neq \varnothing \;\wedge\; \forall p \in P(r)\;\; \exists j \in J(p) : \text{pass}(j) \qquad (1)$$

A requirement is *failed* when some page in *P(r)* is visited only by failing journeys, and *untested* when its row names no page or some page is on no journey. When tests do name *r*, they decide on their own: met when all passed at desktop width, failed otherwise. The result overwrites whatever the builder wrote, with the evidence (which journeys passed through which pages), and is marked as decided by the Studio.

I. Deployment

Deployment is planned first, like a change request (Fig. 4). The targets are Vercel, Netlify, Azure App Service, AWS EC2, AWS ECS and GitHub, filtered by what the stack supports (Table X). For each target the agent reads three skill pages, staged into the record: a core page, the target's own page and the stack's page. The planner works read-only, may ask at most three questions (the account, a choice that costs money, a value only the user has), states every other choice as a default, and looks things up on the web only when the skill pages and the project do not answer them. Code rejects a plan whose author did not open the skill pages with *read_file*. The user approves, revises or cancels it.

TABLE X. DEPLOYMENT TARGETS

Target	Tool	Where it builds	Time	Monitors
Vercel	vercel	Provider's cloud	~10 min	35
Netlify	netlify	Locally, once, before upload	~10 min	8
Azure App Service	az	Artifact made locally	~15 min	12
AWS EC2	aws	Artifact made locally	~25 min	15
AWS ECS	aws	Provider's cloud	~35 min	17
GitHub	gh, git	Repository, CI and optional Pages, checked by a fresh clone	~10 min	16

Time is the core skill page's target from first command to LIVE. Monitors are read-only provider commands the Studio offers after a deployment; every target also gets GitHub's.

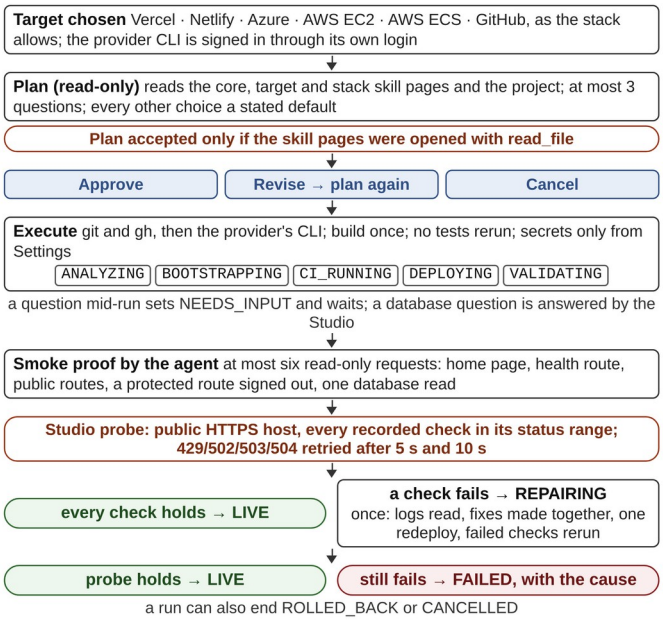


Fig. 4. Deployment flow. The user approves the plan; the agent deploys with the provider's own tool; the Studio probes the address itself and allows one repair round.

The core skill page is a speed contract. A deployment is a delivery, not another test run: no unit, end-to-end, accessibility, performance, security or dependency check is run again; the application is built once, where it is cheapest; only the files a deployment needs are read; the plan is carried out in one pass, slowest provider step first; and a wait for DNS, a certificate or a cluster has a budget of ten to fifteen minutes. Secrets come only from Settings into the environment of the run's own commands, never through the chat or a command line the agent shows. The agent works through each provider's command-line tool, signed in through that tool's own browser or device-code login, so nobody pastes a token. A question that arises mid-run sets the run to *NEEDS_INPUT* and waits; a question about the database connection is answered by the Studio itself, which already holds it.

The run record moves through *ANALYZING*, *BOOTSTRAPPING*, *CI_RUNNING*, *DEPLOYING* and *VALIDATING* and must end on *LIVE*, *FAILED* or *ROLLED_BACK*, with the public address, the provider identifiers, the domain, the database, the names (never the values) of the environment variables, the gaps, the cost and every command that mattered as evidence. Before recording *LIVE*, the agent makes at most six read-only requests: the home page, the health route, up to two public routes, one protected route requested signed out (which must refuse the way the specification says, never with a server error), and one database read through the running application. The Studio then checks the result itself. It requests the public address and every check the agent recorded, and rejects an address that is not HTTPS or whose host is local or a private IP, and any status outside the expected range; 429 and gateway errors from a host that is still starting are retried after 5 s and 10 s. A failure goes back to the agent once: the logs are read once, related fixes are made together, the application is deployed once more and only the failed checks run again. If the address still does not hold, the run ends as failed, with the cause. After a deployment, the Studio offers 97 read-only monitor commands across the providers' command-line tools and a database panel of 22 more, all run on a click and streamed; the panel masks any database value whose field name says it holds a credential.

J. Safety, Privacy and Reliability

Each project runs in one of four permission modes. In *automatic* mode the agent edits files and runs its commands on its own and only risky commands are asked about; in *manual* mode every edit and command is asked about; in *accept edits* mode edits and the usual web-development commands proceed and anything else is asked about; and *bypass* asks nothing and must first be switched on in the menu. A risky command (*sudo*, a download piped into a shell, Invoke-Expression or Start-Process, a credential file sent to a network command) is always asked about and cannot be allowed permanently. Commands are also checked against an allowlist of 132 program names; the code describes this as defence in depth, not a sandbox. The agent may not start a server, since the Studio owns the preview, and may not stop every Node process, since that would stop the Studio. AgentForge's own source is snapshotted before every command and restored if the command changed it.

Personal data does not reach a model in clear text. Everything the user types passes through Microsoft Presidio [31] with a spaCy English model [32], set to 15 entity types: names, e-mail addresses, phone numbers, card and bank numbers, IBANs, IP addresses, crypto wallets, medical licences, several United States and United Kingdom identifiers, and Sri Lankan national identity numbers, for which recognisers were added along with Sri Lankan phone numbers. Each value found is kept, encrypted, in a per-project vault on the user's computer under a placeholder numbered for the whole project (<PERSON_1> is the same person in every message). The one client every model call passes through hides the known values on the way out and restores them in the answer, the reasoning and the arguments of tools that act on the local machine; web search, web fetch and plugin tools keep the placeholders, so a value never leaves the computer through them. Product, business and brand names, the name after "called" or "named", Sri Lankan towns and words the user lists in Settings are not hidden, and a name must be two or more name-shaped words, none of them a word of the trade. If Presidio is not installed, the message is not sent at all, since a filter that silently lets everything through is worse than none. Text shaped like a key, a token or a connection string is refused in the chat and must be saved in Settings instead.

Long runs meet unreliable services. A transient model-service failure is retried up to ten times with back-off, after which a run can hold for up to 30 minutes for the user to press Try again, and Stop ends a model request between tokens and a running command with its process tree. Every stage writes a checkpoint when it starts, finishes or fails, so a run cut off by a restart resumes instead of showing "running" forever. Hosted models are reached with the user's own ollama.com key or through AgentForge's engine proxy, which holds the service key and counts each model call as one request against the limit an administrator sets for the user's account.

K. Human in the Loop

The agents do the work, but the user decides what is built and when a stage moves on. Nothing passes the plan, the specification, the design or a deployment without the user's approval, and every stage has a chat box and an artifact the user

can change (Table XI). Questions are rationed to what only the user can decide: at most 25 in the interview, five before a build, three per change request and three per deployment. Everything else is a stated default that the user can correct.

TABLE XI. WHAT THE USER CAN DO AT EACH STAGE

Stage	User actions
Interview	Pick an answer or type one; attach screenshots and documents; correct the summary; skip
Plan	Approve, or revise in their own words; answer the plan's open questions; each revision is a new version
Specification	Approve; ask for a revision; answer each open decision (the first option keeps the assumption)
Wireframes	Ask in chat; select, draw on or retype any part of a page; approve
Design	Choose 1 of 70 themes or customise: 32 palettes, 117 heading and body fonts, own pictures, a written brief
Prototype	Ask in chat; select, draw on or retype; approve before the build
Build	Connect missing accounts; answer up to five questions; edit code; allow or stop further fix rounds
Test	Choose whether another round of fixes runs; ask for failing journeys to be fixed or rerun
Deploy	Approve, revise or cancel the plan; answer up to three questions; sign in to each provider; save secrets in Settings
After build	Request changes in chat, planned first; select or draw on the running app; open any earlier version

A message typed at any stage is answered from the project or applied to it, and the agent updates every artifact the request reaches (the specification, a diagram, the wireframes, the prototype or the build) without widening it. A change to the specification brings its plain-language overview back in line. A change to the wireframes keeps them low fidelity: a request for colour, a brand look or polish is carried out only as structure and content, and the agent says that styling belongs to the prototype. Once a specification exists, a change is planned first: the planner reads the project, may ask up to three questions and shows a plan card with what will change, in which stage, in what order and in which files; the user approves it, revises it in their own words or cancels it, and nothing is touched before approval. The plan step can be switched off per project. Each approved change becomes a version (v1, v2, ...) that records what was asked, the plan, the files added, changed or removed, and the test results before and after, with a copy of the earlier results, so an update can never cost the project the results it already had.

Users can also point. On the wireframes, the prototype and the running application, a select tool picks an element and a pencil marks an area; the Studio describes what was picked or drawn round (tag, words, attributes, place on the page and a slice of markup) and attaches a screenshot for the agent. A text tool changes words in place: when they occur once in the source, the Studio swaps them and rebundles the app with no model call; otherwise it hands the change to the agent, which can follow the words to wherever they are built. Code can be edited directly in the Studio's code pane.

The user also controls a run while it happens. The permission mode decides what the agent may do unasked, and a permission card offers allow, allow without asking again for this session, or deny with a note for the agent. Further rounds of fixes are a Yes or No card, and a typed yes in English or Sinhala counts. The model and its reasoning effort can be changed per project without losing the project's record, and a model picker marks which models can see pictures.

IV. IMPLEMENTATION

AgentForge Studio 1.0.3 has about 32,300 lines of Python in the backend, the engine and the agents, 27,800 lines of JavaScript in the Studio, and 19,500 lines of Markdown prompts, including the 70 design themes and the deployment skill pages. Table XII lists the technology at each layer. Three choices shape the rest. The backend has no web framework; it runs on the Python standard library, so the desktop build carries an embedded interpreter and seven Python packages. Every model call goes through one client, which is where retries, account metering and the privacy filter live. And every artifact is a file in the project record, so any stage can be stopped, inspected and resumed.

TABLE XII. TECHNOLOGY STACK

Layer	Technologies	Used for
Studio	Next.js 16, React 19, Tailwind CSS 4, Zustand 5, Lucide, JSZip	Chat, stage views, previews, testing and deployment panels
Desktop	Electron 44, electron-builder 26, agentforge:// scheme, C# online installer	One window; backend over stdio
Backend	Python 3.10+ standard library (ThreadingHTTPServer, RFC 6455 feed, threads), Pydantic 2, httpx, cryptography (Fernet)	API, jobs, event bus, schema, vault
Engine	Ollama Python SDK, ten tools, MCP client (JSON-RPC 2.0 over stdio)	Tool-calling loop of every agent role
Models	Local Ollama; ollama.com; Cloudflare Worker proxy with accounts	User-chosen model; vision detected per model
Privacy	Microsoft Presidio, spaCy en_core_web_sm	15 entity types hidden from models
Documents	Mermaid CLI; marked and Puppeteer	SVG diagrams; specification and test-report PDFs
Wireframe and prototype kit	Vite 5, React 18, TypeScript 5, Tailwind 3.4, shadcn/ui (Radix), Recharts, vite-plugin-singlefile	One self-contained bundle.html per app
Browser	Playwright Chromium; Edge or Chrome in headless mode	Screenshots, page inspection, journey walk
Generated apps	Next.js 15; Remix 2 and Vite 6; Express 4; Supabase (supabase-js 2, @supabase/ssr); MongoDB with Mongoose 8; bcryptjs, jose	Eleven stack templates
Generated tests	Vitest, Testing Library, Supertest, Playwright 1.62, axe-core, Lighthouse CI, ZAP, npm audit, GitHub Actions	Unit, journey, visual, a11y, performance, security
Cloud	Supabase OAuth; MongoDB Atlas CLI; gh, Vercel, Netlify, Azure and AWS CLIs	Provisioning, sign-in, deployment
Integrations	20 providers: payments, e-mail and SMS, image storage, social sign-in, maps, analytics	Optional features, keys encrypted

A. Backend Runtime

The API is a ThreadingHTTPServer bound to 127.0.0.1:7824, and the live feed is an RFC 6455 WebSocket server on port 7825 written on the standard library; the Studio proxies */__agentforge/api* and */__agentforge/ws* to them. In the desktop app nothing listens: the window and the backend exchange one JSON object per line over standard input and output, and the event feed travels the same way. The only port the desktop app ever opens is a short-lived listener for a

browser sign-in's redirect, closed as soon as the callback arrives. Work that needs a model never runs inside a request. It becomes a background job on its own thread, and the browser polls for the result, so a specification that takes ten minutes holds no connection open. Every event a stage emits goes to an in-memory bus and is appended to *events.jsonl*, so a browser that reconnects replays the same conversation. The project's conversation is written to *context.json* after every turn, turns removed by compaction are appended verbatim to *context-archive.jsonl*, and a checkpoint at the start, end or failure of every stage lets an interrupted run resume.

B. Engine Logic

One request to the engine is a loop of at most 40 model turns. Each turn sends the conversation with the tool schemas (without the write and command tools in plan mode), executes the returned tool calls and appends their results. Calls that write different files are run up to four at a time, which the wireframe and prototype stages use; edits and commands stay in order. A turn ends the loop when it carries no tool call or a completion marker; a plain closing sentence such as "the task is complete" is accepted when a model omits the marker, unless it also says the work is incomplete. A tool result is capped at 24,000 characters and a file read at 32,000, and reading a *.env* file is hidden from the activity log.

Context is estimated at four characters per token plus 1,500 per picture. Past 88% of the window, older turns are cut into stretches of up to 100,000 characters, summarised four at a time and merged in groups until one memory remains; a stretch the model will not summarise becomes a mechanical digest of the requests, commands and file paths it contains. When the service rejects a request as too long, the token count it reports rescales the estimate and the history is compacted again, up to three times. A screenshot is attached only while the whole request stays under 2.4 MB, and older pictures are replaced by a note. Structured answers go through a validator, whose error is sent back for at most three attempts, and repairs to the specification are JSON edits (*set*, *add*, *remove*) addressed by path and item key, such as *functional_requirements[id=FR-012]*.

Independent work runs in parallel lanes: five specification parts at once, six diagrams, four compaction workers, and three agents for wireframe pages, page phases and test phases, each agent in a conversation of its own that is archived when it ends. Three is close to the number of concurrent requests the hosted model service accepts. The interview's bookkeeping of facts and coverage runs on a background thread while the user reads the next question, and the specification's diagram review and its prose build brief are written after the stage has been handed over. Algorithm 1 shows how the build combines these parts.

Algorithm 1 Phased build with verified phases

Input: approved routes R, stack s, the user's direction d, build request q
$P \leftarrow$ [foundation] + chunks(R, 4) + [wiring, unit tests, journey tests]
$f \leftarrow$ SHA-256(R, s, d); if a saved state has fingerprint f, keep its done phases
for each phase $p \in P$ not done **do**
  if p starts a run of independent phases **then** build the run in 3 parallel lanes
  else build p in a fresh conversation that begins with q
  repeat at most twice **while** problems(p) $\neq \varnothing$: repair(p, problems(p))
  mark p done if problems(p) = $\varnothing$, else incomplete; save state
if some $r \in R$ has no recorded, non-empty file **then fail** naming r
review pages; run journeys one by one; run final checks; set requirement status by Eq. (1)

C. Verification Logic

A page phase is checked against its own progress record, *build/progress/<phase>.json*, which must name files that exist inside the workspace and are not empty. The Studio's journey run is keyed by a SHA-256 fingerprint over the path and content of every source file, skipping dependencies, build output, test output and the record: while the fingerprint is unchanged, a later Testing run reuses the result instead of running again, and a stopped build that carries on repeats none of the testing already done on the application as it is. The runner script, which the Studio refreshes in every project before a run, builds the app once, starts the production server on a free port, runs Playwright with one retry and two workers, and stops only the server it started; on MongoDB stacks it creates a database named within Atlas's 38-character limit and ending in *_e2e*, and drops it afterwards. The suite has 45 minutes, the page photographs 20, the runtime check 10 and the build 15, and screenshots are copied out as the tests take them, because the next run empties Playwright's output folder.

The specification checks are plain functions. Routes are compared by a key that ignores letter case and parameter names, so */Admin/[branchId]* and */admin/:id* are one page; record names are matched to tables through singular and plural forms. The evidence audit scores 100 times the share of its seven rules that pass, capped at 50 when either requirement set is missing, and any score below 100 adds blocking findings. The build and QA reports are filled from a template and checked against it, with at most two repair rounds. The deployment planner must have opened the target's skill pages with *read_file* before its plan is accepted, and the live probe accepts a status range written as 200, 2xx or 200–399.

D. Safety Mechanisms

A command is split on &&, ||, *;* and | outside quoted strings, and the program of each part is checked against the allowlist; PowerShell cmdlets pass by their Verb-Noun shape except six that run code or another process, and PowerShell keywords, variables and literals are not mistaken for programs. A command may run 600 s, a deployment release 1,800 s and a server start 75 s; on Windows every process starts without a window. Text search skips dependencies, build and test output and binary files and stops after 20 s. Before each command the files of AgentForge itself are snapshotted, and afterwards any change is undone unless the repository's HEAD moved, which marks a deliberate pull or checkout.

E. Privacy and Model Access

Presidio analyses typed text in 50,000-character chunks and keeps matches scoring at least 0.4; phone numbers are recognised for ten regions. Values go to a Fernet-encrypted vault under placeholders such as <PERSON_1>, and a deleted project's values go with it. The model client replaces known values in every message, tool result and earlier tool call before sending, and restores them in the reply; in a streamed reply it holds back a chunk that ends with what may be the start of a placeholder until the next chunk completes it. A name is hidden only as a whole word and never inside code. Provider keys for the 20 optional integrations are encrypted the same way.

The client retries HTTP 408, 425, 429, 500, 502, 503, 504, 522 and 524 and broken transports up to ten times, waiting 2, 4, 8, 15, 20 and then 30 s, and Stop is checked every 0.1 s, so a run never waits for a reply it no longer wants. Replies from hosted models are streamed, since a single response is cut off by the service's edge after about 100 s without a byte. Whether a model can see pictures is read from its own capability list and cached for a week. The engine proxy is a Cloudflare Worker that compares the app token in constant time, rate-limits each caller, passes through eight routes (chat, generate, model details and list, version, web search, web fetch and balance), and doubles as the OAuth broker for Supabase sign-in, so its client secret never ships with the app. Each model call is counted as one request against the limit an administrator sets for the account, from the moment a streamed reply starts.

F. Distribution and Tests

The release script exports the Studio as static files, stages the backend with its Node modules and an embedded Python 3.12, packages Electron, pins the command-line tools, and compiles a 150 KB online installer with the C# compiler built into Windows; the installer checks every file against a SHA-256 manifest. The desktop app serves the Studio from an *agentforge://* scheme, and every process it starts is its child, so quitting ends them all. Models are chosen by the user: the bakery project in Section V-B used gpt-oss:120b through Ollama's cloud, and accounts default to DeepSeek V4.1 Flash. The repository holds 1,733 Python unit tests and twelve Node test files, which cover the engine, every gate, the privacy filter and vault, the event shapes the Studio reads and the deployment flow, with stand-in models instead of live calls.

V. Evaluation and Preliminary Results

A. Research Questions and Protocol

The observations in Section V-C were gathered while the system was being built, so they cannot attribute an improvement to one component. The controlled study is the next step. A fixed suite of application ideas, from a one-screen tool to stores of more than thirty screens, is built with the same model, sampling settings and context length under each condition of Table XIII. Each task runs three times per condition and results are reported as mean and standard deviation. Model requests are logged at the single client all calls pass through, which gives exact token counts and makes leaked personal data countable. Requirement verdicts are compared with a manual test by a tester who has not seen the system's verdict.

TABLE XIII. Research Questions, Conditions and Metrics

Research question	Condition	Primary metrics
RQ1: Do the coverage gate and evidence audit make the specification more complete than the first draft?	Same plan; gates on vs. first draft kept	Plan items with no requirement; audit score; omissions found by two raters
RQ2: Does a phased build with a named-page gate deliver every approved page?	Phased vs. one conversation, same prototype	Approved pages built; wall-clock time; tokens
RQ3: How far do agent-reported requirement statuses diverge from Eq. (1)?	Builder report vs. Eq. (1) vs. blind manual test	Cohen's κ with the human verdict; false "met"
RQ4: What do per-phase conversations cost or save?	Fresh per phase vs. one shared conversation with compaction	Tokens and latency per call; total time; defects from lost context
RQ5: How reliably does plan-first deployment reach a probed live address?	Each finished app to each target its stack allows	Success rate; time against target; repair rounds; questions asked
RQ6: Does the privacy filter keep personal data out of model requests, and at what cost?	Ideas seeded with synthetic personal data; filter on vs. off	Seeded values in logged requests; terms wrongly hidden; artifact defects
RQ7: How much user effort does a project take?	Each task, instrumented Studio	Questions answered; approvals; chat edits; minutes of user time

B. Development Corpus

The observations come from the authors' own projects built during development: a bakery ordering application taken from interview to prototype, a 28-page application taken from specification to deployment, a 31-page online store, and a store whose plan had 71 screens. Each row of Table XIV is one observed run before and, where stated, after a change, as recorded in the code and its version history. Together they describe how the pipeline failed and why it is built the way it is; they are not a comparison with other systems.

C. Observations

Table XIV lists 23 failures in the order of the pipeline. The coverage gate gave the earliest result: on the bakery project it took the first specification draft from one diagram to seven and from three functional requirements to six. The largest costs were in time. Before the changes in rows 6, 8, 9 and 20, the 28-page application took about seven hours of machine time from specification to deployment.

TABLE XIV. Failures Observed During Development and the Change Each Led To

Failure observed	Measurement	Change made
1. Each interview turn rewrote facts, coverage and assumptions (~4,000 tokens) first	~20 s per question (up to 64 s); after: 4.6, 2.0, 1.9, 1.9 s	Question asked alone; bookkeeping in the background
2. Specification as one ~60,000-token JSON reply	546 s for a 31-page store; after: 141 s	Five parts in parallel, joined, validated
3. Build brief written before the specification was handed over	The last 36 s of every specification	Written in the background
4. Routes compared with letter case (the filter capitalised "admin")	117 pages for 71 screens; 46 duplicates	Case- and parameter-insensitive route keys
5. Name recogniser took "Admin", "Docker", "Require" for people	A 117-page wireframe ended with 2 pages	Names need two name-shaped words; whole words only
6. One agent drew every wireframe page	About 1.5 h for 117 pages	Shared layout and 3 pages, then 3 agents

Failure observed	Measurement	Change made
7. Compaction summarised one stretch after another	Session silent for about 15 min on a long project	Four stretches at a time
8. Build began inside the earlier stages' conversation	672,000 tokens resent with every build call	Every stage starts from a small conversation
9. One conversation through every build phase	Up to 900,000 tokens; 21 s per call vs. 5 s at 50,000	Fresh conversation per phase
10. One conversation asked to build the whole app	A coherent subset; the model cited "capacity"	Numbered phases; named-page gate
11. One agent built every page, four per phase	2 h 37 min for 31 pages	Three agents on page phases
12. MongoDB not connected, found mid-build	Seed and every journey failed hours later	Preflight card before the plan
13. Agent ran the whole suite through Playwright's own script	Six full runs in one build	Command refused; the Studio runs it once
14. All journeys run first, then fixed together	About 1.5 h on a 31-page store	One journey at a time, fixed at once
15. Phone page photographed at device pixel scale	1082 × 10973 px, too tall for a model to read	One pixel per CSS pixel
16. Builder marked requirements before tests ran	0 of 116 met after all tests passed	Status from runner output, Eq. (1)
17. A fix broke the prototype's bundle during the journey walk	The walk ended at a bundler error	Bundle repaired; the walk goes on
18. Screenshot added to a long conversation	2.5 MB answered; 3.4 MB got HTTP 500	Pictures attached only below 2.4 MB
19. Long hosted reply sent as one response	Cut off by the service edge after ~100 s (HTTP 524)	Replies streamed
20. Text search walked test frames and packages	16 min for one search (58,000 frames); deployment plan 37 min	Skip outputs and binaries; 20 s cap
21. Deployment planner searched the web for what its skill pages said	Six searches and two page reads in one plan	Search only when skills and project do not answer
22. Release command killed at the 10-minute limit	A half-made deploy and stale build output	Releases may run 30 minutes
23. Element picker in the built app's preview did nothing	No bridge in the preview's own pages	Preview hook injects the bridge

D. Discussion

Shared files beat a shared conversation. The first design treated the conversation as the project's memory. It failed on cost rather than correctness (rows 7 to 9 and 18): by the build, every call resent hundreds of thousands of tokens, a long history took a quarter of an hour to compact, and a request that also carried a screenshot exceeded what the service accepted. Compaction kept the conversation inside the window but did not make it fast. The switch to small conversations was safe because every stage's input was already a file: the plan, the specification, the hand-off files and the per-phase progress records. A fresh phase reads those and loses nothing it needs, and the verbatim archive keeps what it does not. This fits the evidence that models use long inputs unevenly [19].

Self-reports are the weakest evidence. Rows 10 and 16 are one failure in two stages: the model's account of its work differed from the files. In row 10 the model shortened the application on its own judgement; in row 16 it marked requirements before a test had run. Each fix replaced a statement by the model with a function of what is on disk, consistent with the finding that models do not reliably correct themselves without external feedback [17]. The specification reviewer's verdict is recorded but not obeyed for the same reason, and the deployment agent's claim of a live address is checked by a request the Studio makes itself.

A check that fires wrongly does damage. The privacy filter's false positives did not merely hide words; they rewrote code (row 5). With "require" taken for a name, the agent read <PERSON_3>("fs") and wrote it back into the files. One capital letter in a route, introduced by that same filter, produced 46 duplicate pages (row 4). A check that rewrites content needs precision more than recall, and both checks were narrowed until they fired only on the case they were written for.

Ask early, ask once. Rows 12 and 13 cost hours because a missing service was discovered in the middle of the run and because an agent kept repeating the most expensive step on its own. Moving every external dependency into one preflight card, every decision the model cannot make into at most five questions before the plan, and every further round of fixes into a Yes or No card turned open-ended loops into decisions the user makes once. The same reasoning sets the speed contract of deployment, where a second test campaign or a third repair round is not allowed.

Parallelism has a ceiling. Lanes cut the long stages (rows 2, 6 and 11), but three agents is close to what the hosted service accepts at once, and phases built in parallel must not write the same file, which is why each records its pages in its own progress file. Work that depends on an earlier result, such as a journey whose fix changes a shared helper, stays sequential (row 14).

VI. Limitations and Future Work

Seven limitations bound these claims. First, the evidence is internal and uncontrolled: single runs on the authors' projects, no baseline, and no attribution of an improvement to one component; the study of Section V-A is the next step. Second, the coverage gate matches plan items to requirements by shared words, so a requirement that mentions a feature without specifying it passes, and the evidence audit checks the form of a requirement (an identifier, "shall", a unit), not its truth. Third, Eq. (1) is only as strong as the journeys: a journey that visits a page but asserts little still marks that page's requirements met, and the mapping works at page granularity rather than at the level of acceptance criteria.

Fourth, the screenshot reviews, the diagram review and the journey walk need a vision-capable model; without one they are skipped, and a person still judges how the product looks. Fifth, privacy has known gaps: the screenshots shown to a vision model carry the real personal data the filter hides everywhere else, files the user attaches are read as they are, and name detection uses an English model, although the patterns for e-mail addresses, phone and identity numbers work in any language. Sixth, the system targets JavaScript web applications on Supabase or MongoDB, ships for Windows, and its command checks are defence in depth rather than isolation. Seventh, the stages depend on a hosted model service for long runs, and its limits (requests in flight, request size, response time) shaped several design choices that a different service might not need.

Future work follows from these. The controlled study will measure the effect of each check. Journey assertions will be tied to acceptance criteria rather than pages, so that a requirement is met only when the behaviour it names is observed. Agent commands will run inside a container, and the stacks will extend beyond JavaScript.

VII. Conclusion

AgentForge started from the view that the hard part of autonomous software engineering is not writing code but knowing when it is done. The system hands that decision to code at every boundary: a depth floor on the plan, coverage and evidence checks on the specification, a named-page gate on the build, requirement status computed from test results, and a deployed address checked by the Studio rather than by the agent that deployed it. It leaves what to build, and when to move on, to the user, who approves each major stage and can change any of them by chat or by pointing at the page. Building it showed two things the first design did not expect. A conversation is a poor place to keep a project, and small per-phase conversations over a shared record made long builds tractable. And every place where a model's report stood in for evidence eventually gave a wrong answer that a short function over the files could correct. The controlled study of Section V-A will test whether these mechanisms hold beyond the projects they were built on.

References

[1] M. Chen et al., "Evaluating large language models trained on code," arXiv:2107.03374, 2021.

[2] B. Hui et al., "Qwen2.5-Coder technical report," arXiv:2409.12186, 2024.

[3] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[4] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2024.

[5] J. He, C. Treude, and D. Lo, "LLM-based multi-agent systems for software engineering: Literature review, vision, and the road ahead," *ACM Trans. Softw. Eng. Methodol.*, 2025, doi: 10.1145/3712003.

[6] Gemma Team, "Gemma 3 technical report," arXiv:2503.19786, 2025.

[7] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[8] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying LLM-based software engineering agents," arXiv:2407.01489, 2024.

[9] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021.

[10] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Chichester, U.K.: Wiley, 2009.

[11] C. Qian et al., "ChatDev: Communicative agents for software development," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2024.

[12] S. Hong et al., "MetaGPT: Meta programming for a multi-agent collaborative framework," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[13] Q. Wu et al., "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155, 2023.

[14] D. Huang, J. M. Zhang, M. Luck, Q. Bu, Y. Qing, and H. Cui, "AgentCoder: Multi-agent-based code generation with iterative testing and optimisation," arXiv:2312.13010, 2023.

[15] A. Madaan et al., "Self-Refine: Iterative refinement with self-feedback," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023.

[16] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023.

[17] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, "Large language models cannot self-correct reasoning yet," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[18] L. Zheng et al., "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets and Benchmarks Track*, 2023.

[19] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Trans. Assoc. Comput. Linguistics*, vol. 12, pp. 157–173, 2024.

[20] *Systems and Software Engineering—Life Cycle Processes—Requirements Engineering*, ISO/IEC/IEEE 29148:2018, 2018.

[21] A. Ferrari, G. O. Spagnolo, and S. Gnesi, "PURE: A dataset of public requirements documents," in *Proc. IEEE 25th Int. Requirements Eng. Conf. (RE)*, 2017, pp. 502–505.

[22] M. Lima, V. Valle, E. Costa, F. Lira, and B. Gadelha, "Software engineering repositories: Expanding the PROMISE database," in *Proc. 33rd Brazilian Symp. Softw. Eng. (SBES)*, 2019.

[23] P. Giannouris, "Multi-agent LLM system for UML class diagram generation from natural language requirements," arXiv:2511.22409, 2025.

[24] Ollama. [Online]. Available: https://ollama.com

[25] Anthropic, "Model Context Protocol specification." [Online]. Available: https://modelcontextprotocol.io

[26] Anthropic, "web-artifacts-builder," Agent Skills repository. [Online]. Available: https://github.com/anthropics/skills

[27] Microsoft, "Playwright." [Online]. Available: https://playwright.dev

[28] Deque Systems, "axe-core: Accessibility engine for automated web UI testing." [Online]. Available: https://github.com/dequelabs/axe-core

[29] Google, "Lighthouse." [Online]. Available: https://developer.chrome.com/docs/lighthouse

[30] ZAP by Checkmarx (formerly OWASP ZAP), "Zed Attack Proxy." [Online]. Available: https://www.zaproxy.org

[31] Microsoft, "Presidio: Data protection and de-identification SDK." [Online]. Available: https://github.com/microsoft/presidio

[32] M. Honnibal, I. Montani, S. Van Landeghem, and A. Boyd, "spaCy: Industrial-strength natural language processing in Python," 2020, doi: 10.5281/zenodo.1212303.